



Curso:

Disciplina: Programação de Sistemas Web

Professor: Lucas de Oliveira Teixeira

Aluno: \_\_\_\_\_

Data: \_\_\_\_\_

R.A.: \_\_\_\_\_

## Trabalho - Parte 1 - Frontend Interativo

### 1. Introdução

Esta especificação descreve a **Parte 1** do projeto integrador da disciplina de **Programação de Sistemas Web**. Os conteúdos abordados em aula (HTML, CSS, JavaScript e DOM) usam exemplos extraídos deste projeto.

O objetivo é desenvolver uma aplicação web funcional chamada **CineTrack**: um gerenciador pessoal de listas de filmes. Nesta parte, toda a lógica reside no navegador, sem servidor e sem banco de dados; os dados são mantidos localmente. Em uma segunda parte, tratada em documento próprio, o mesmo projeto evolui para uma aplicação full stack com React e um servidor Node.js.

O trabalho é construído de forma incremental: cada etapa acrescenta um trecho concreto ao projeto, versionado com *commits* ao longo do desenvolvimento. A versão final desta parte é marcada no repositório pela tag *v1.0*.

Esta especificação descreve o comportamento esperado da aplicação, não uma implementação única. Desde que os requisitos sejam atendidos, diferentes organizações de código, nomes e estratégias são aceitos.

### 2. Descrição Geral do Sistema

O **CineTrack** é um aplicativo web que permite ao usuário gerenciar sua lista pessoal de filmes. Para cada filme cadastrado, são armazenadas as seguintes informações:

- **Título**: nome do filme.
- **Ano**: ano de lançamento (número inteiro).
- **Gênero**: categoria do filme (ex.: Ação, Drama, Comédia).
- **Pôster**: URL de uma imagem de capa.
- **Nota**: avaliação do usuário, de 1 a 5 estrelas.
- **Status**: situação do filme na lista do usuário: *assistido*, *assistindo* ou *quero assistir*.
- **Comentário**: campo de texto livre com a opinião do usuário (opcional).

A interface principal exibe os filmes em uma grade de cards visuais. O usuário pode buscar filmes por título, filtrar por status, adicionar novos filmes, editar os existentes e removê-los da lista.

#### 2.1 Fluxo básico de uso

1. O usuário acessa a página principal e visualiza sua lista de filmes em cards.
2. Por meio da barra de busca, é possível filtrar filmes cujo título contenha o texto digitado.
3. Os botões de filtro permitem exibir apenas filmes de um determinado status.
4. Ao clicar em “+ Adicionar”, um formulário de cadastro é aberto.
5. Após o preenchimento e envio do formulário, o novo card aparece na lista sem recarregar a página.
6. Cada card oferece botões de edição e remoção do filme.



### 3. Requisitos

**Tecnologias:** HTML5 · CSS3 · JavaScript (ES6+) · DOM API · persistência local no navegador

Nesta etapa, toda a lógica da aplicação reside no navegador. Os dados são mantidos em memória e persistidos localmente, de modo que a lista do usuário seja preservada ao recarregar a página; na primeira visita, carregam-se os dados de exemplo da Seção 5. Não há servidor nem banco de dados.

#### 3.1 Requisitos Funcionais

Os requisitos abaixo descrevem **o que** a aplicação deve fazer. A forma de implementar cada um fica a critério do aluno.

##### 3.1.1 RF-01: Lista de filmes em cards

A página principal deve exibir os filmes cadastrados em uma grade responsiva de cards, gerada dinamicamente pelo JavaScript a partir dos dados em memória. Cada card deve apresentar:

- Imagem do pôster, com texto alternativo descritivo (usar a URL fornecida ou uma imagem de *placeholder* quando ausente);
- Título do filme em destaque;
- Ano de lançamento e gênero;
- Nota visual em estrelas (sempre 5 símbolos: cheios e vazios);
- Indicação visual do status (*badge* colorida distinta para cada status);
- Botões de **editar** e **remover**.

##### 3.1.2 RF-02: Busca por título

Um campo de busca deve filtrar os cards exibidos de acordo com o que o usuário digita. A busca deve ser **case-insensitive** e reagir enquanto o usuário digita, atualizando a lista sem recarregar a página. Recomenda-se aplicar a técnica de *debounce* para evitar re-renderizações a cada tecla.

##### 3.1.3 RF-03: Filtro por status

A interface deve oferecer botões que filtram a visualização por status (*Todos*, *Assistido*, *Assistindo* e *Quero assistir*). O botão ativo deve ter destaque visual e a opção *Todos* começa selecionada. O filtro de status deve funcionar em conjunto com a busca por título: os dois critérios se combinam.

##### 3.1.4 RF-04: Cadastro de filme

A aplicação deve oferecer um formulário para adicionar filmes, com validação nativa do HTML. O formulário deve conter os seguintes campos, cada um com rótulo (`<label>`) associado:

- Título (texto, obrigatório);
- Ano (número, obrigatório, com faixa de valores válida);
- Gênero (texto, obrigatório);
- URL do pôster (texto, opcional);
- Status (seleção com as três opções, obrigatório);
- Nota (número de 1 a 5, obrigatório);
- Comentário (texto longo, opcional).

O acionamento do cadastro deve abrir o formulário limpo. Se o formulário for apresentado em uma janela sobreposta (modal), ela deve poder ser fechada tanto por um botão de cancelar quanto pela tecla `Esc`.



### 3.1.5 RF-05: Validação do formulário com JavaScript

No envio do formulário, os dados devem ser validados por JavaScript antes de serem salvos. Em caso de erro, as mensagens devem ser exibidas na interface (não usar `alert()`) e o filme não é salvo. Os campos numéricos devem ser tratados como números.

### 3.1.6 RF-06: Edição de filme

Ao acionar **Editar** em um card, o formulário deve ser reaproveitado, pré-preenchido com os dados do filme selecionado. Ao confirmar, o filme correspondente é atualizado e a lista é re-renderizada.

### 3.1.7 RF-07: Remoção de filme

Ao acionar **Remover**, o usuário deve confirmar a ação antes de o filme ser excluído. Após a confirmação, o filme sai da lista e a exibição é atualizada. Os botões dos cards devem continuar funcionando.

### 3.1.8 RF-08: Persistência local

A lista de filmes deve ser persistida localmente no navegador, de modo que seja carregada ao abrir a página e regravada após cada adição, edição ou remoção. Recarregar a página não pode perder dados.

### 3.1.9 RF-09: Dados iniciais pré-carregados

Na primeira visita (sem dados salvos), a aplicação deve carregar os **6 filmes de exemplo** da Seção 5. Cada novo filme adicionado deve receber um identificador único.

## 3.2 Requisitos Não Funcionais

- **Layout responsivo:** a página deve ser utilizável de 320 px (celular) a 1920 px (desktop). A grade de cards deve usar CSS Grid; o cabeçalho e os filtros podem usar flexbox; media queries ajustam os breakpoints.
- **Tema com variáveis CSS:** cores e raios centralizados em variáveis CSS; recomenda-se dark mode automático via `prefers-color-scheme`.
- **Acessibilidade:** rótulos em todos os campos, texto alternativo nas imagens e uso adequado de atributos ARIA na busca e na navegação. A aplicação deve ter boa pontuação de acessibilidade.
- **Sem frameworks JS:** não é permitido jQuery, React ou qualquer outra biblioteca JavaScript nesta parte. Apenas JavaScript puro (ES6+).
- **Organização do código:** HTML, CSS e JavaScript em arquivos separados.
- **Compatibilidade:** a aplicação deve funcionar nos navegadores modernos (Chrome, Firefox, Edge, versões recentes).

## 3.3 Estrutura de Arquivos Sugerida

A estrutura abaixo é uma sugestão. O essencial é que o código-fonte da Parte 1 fique organizado na pasta `parte1/`, com HTML, CSS e JavaScript em arquivos separados.

```
cinetrack/  
|-- README.md  
|-- .gitignore  
|-- docs/  
'-- parte1/  
    |-- index.html  
    |-- style.css  
    '-- app.js
```

Listing 1: Estrutura sugerida na tag v1.0 (Parte 1)



### 3.4 O que será avaliado

A avaliação considera, de forma qualitativa:

- Estrutura HTML semântica e acessível;
- Layout com CSS: grade responsiva, tema e estados visuais dos elementos interativos;
- Renderização dinâmica dos cards a partir dos dados, via DOM;
- Busca por título e filtro por status funcionando em conjunto;
- Cadastro, edição e remoção de filmes, com validação e confirmação;
- Persistência local dos dados entre recarregamentos;
- Organização do código e histórico de *commits*.

### 3.5 Entrega

**Formato de entrega:** link do repositório Git do projeto *cinetrack*, com a tag *v1.0* criada (`git tag v1.0 && git push -tags`).

**O que deve ser entregue:**

- Código-fonte completo da Parte 1 em *parte1/* (HTML, CSS e JavaScript);
- Arquivo *README.md* com: nome(s) do(s) aluno(s), RA(s) e instruções para executar o projeto.

## 4. Regras Gerais

### 4.1 Formação de grupos

O projeto pode ser realizado **individualmente ou em dupla**. Não serão aceitos grupos com mais de dois integrantes. Grupos formados devem ser informados ao professor até a data indicada no cronograma da disciplina; alterações posteriores não serão permitidas.

### 4.2 Autoria e plágio

- Cada grupo deve produzir seu próprio código. É permitido consultar documentações, tutoriais e exemplos da internet, desde que o código entregue seja escrito e compreendido pelo grupo.
- **Plágio** (copiar código de outro grupo ou de repositórios públicos de outros alunos da disciplina) resultará em nota **zero** para ambos os grupos envolvidos, sem possibilidade de recurso.
- Ferramentas de inteligência artificial generativa (ex.: GitHub Copilot, ChatGPT) podem ser utilizadas como auxílio, mas o grupo é responsável por compreender e explicar todo o código entregue. O professor pode realizar arguição oral sobre qualquer parte do projeto.

### 4.3 Formato do repositório Git

- As duas partes do trabalho devem permanecer no **mesmo repositório**. A Parte 1 fica preservada na pasta *parte1/*.
- A tag *v1.0* é **obrigatória** e marca a versão avaliada desta parte.
- O repositório deve ser **público** (ou o professor deve ser adicionado como colaborador).
- O histórico de *commits* será considerado na avaliação: projetos com um único *commit* contendo todo o código serão penalizados.
- Usar mensagens de *commit* no padrão Conventional Commits (*feat:*, *fix:*, *docs:*), com descrição em português.



## 5. Dados de Exemplo para Testes

Os 6 filmes abaixo são o conjunto de dados canônico da disciplina: os mesmos dos slides e das atividades. Na Parte 1, eles são os dados iniciais pré-carregados na primeira visita.

```
[
  {
    "id": 1,
    "titulo": "A Origem",
    "ano": 2010,
    "genero": "Ficção científica",
    "poster": "https://placeholder.co/200x300?text=A+Origem",
    "nota": 5,
    "status": "assistido",
    "comentario": "Revejo sempre."
  },
  {
    "id": 2,
    "titulo": "Parasita",
    "ano": 2019,
    "genero": "Suspense",
    "poster": "https://placeholder.co/200x300?text=Parasita",
    "nota": 4,
    "status": "assistido",
    "comentario": ""
  },
  {
    "id": 3,
    "titulo": "O Auto da Compadecida",
    "ano": 2000,
    "genero": "Comédia",
    "poster": "https://placeholder.co/200x300?text=Compadecida",
    "nota": 5,
    "status": "assistido",
    "comentario": "Classico nacional."
  },
  {
    "id": 4,
    "titulo": "Duna: Parte Dois",
    "ano": 2024,
    "genero": "Ficção científica",
    "poster": "https://placeholder.co/200x300?text=Duna+2",
    "nota": 4,
    "status": "assistindo",
    "comentario": ""
  },
  {
    "id": 5,
    "titulo": "Interestelar",
    "ano": 2014,
    "genero": "Ficção científica",
    "poster": "https://placeholder.co/200x300?text=Interestelar",
    "nota": 5,
    "status": "quero",
    "comentario": ""
  }
]
```



```
},  
{  
  "id": 6,  
  "titulo": "Cidade de Deus",  
  "ano": 2002,  
  "genero": "Drama",  
  "poster": "https://placeholder.co/200x300?text=Cidade+de+Deus",  
  "nota": 4,  
  "status": "quero",  
  "comentario": ""  
}  
]
```

Listing 2: Dados de exemplo

### 5.1 Notas sobre os dados de exemplo

- Os pôsteres usam o serviço `placeholder.co`; troque por URLs de imagens reais (ex.: TMDB) se desejar.
- O campo "status" assume os valores "assistido", "assistindo" e "quero" (o rótulo exibido para o último é "Quero assistir").
- O campo "id" é um número inteiro único.

*Bom trabalho e bons estudos!*